PROJETO DOAAÇÃO

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP 1º BIMESTRE)

Instituição: Unicesumar | **Curso:** Engenharia de Software
Turma: esoft4sm | **Período:** 4º semestre — 2026.2

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

Grupos solidários podem receber doações por diferentes canais e depender de anotações dispersas. Isso dificulta identificar os itens recebidos, a instituição destinatária e o que já foi entregue. O **DoaAção** propõe centralizar esses registros em uma aplicação local, utilizada pelo voluntário responsável pela organização das doações.

O projeto relaciona-se à **ODS 2 da ONU (Fome Zero e Agricultura Sustentável)** pela organização das doações de alimentos. O registro de produtos de higiene complementa a ação social. O objetivo é facilitar o acompanhamento dos alimentos arrecadados e de sua destinação às instituições.

Concepção e jornada do usuário
A proposta considera um cenário de organização de doações por voluntários. A jornada do usuário é: registrar a doação, incluir os itens, consultar os registros e atualizar a situação para ENTREGUE. Os dados permanecem no banco local após fechar o aplicativo.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

- **RF01** - Cadastrar uma doação com doador ou grupo, instituição destinatária e situação RECEBIDA.
- **RF02** - Incluir um ou mais itens com tipo, descrição, quantidade inteira positiva e unidade; para alimentos, registrar também a validade.
- **RF03** - Listar as doações e consultar os dados e itens de uma doação selecionada.
- **RF04** - Alterar doador, instituição, situação e os itens de uma doação existente.
- **RF05** - Excluir uma doação após confirmação, removendo também seus itens.
- **RF06** - Registrar e exibir as situações RECEBIDA e ENTREGUE.
- **RF07** - Exibir o total de doações e a contagem por situação.

3. O PLANEJAMENTO ÁGIL (BACKLOG)

O desenvolvimento está organizado em quatro etapas, seguindo a dependência entre modelagem, implementação e validação. O backlog prioriza o cadastro, a consulta, a edição e a exclusão de doações, seguidos pelo acompanhamento das entregas.

Etapa	Épico	User Story / Atividades
Etapa 1	Concepção e modelo	**COMO UM** voluntário **EU QUERO** um registro organizado **PARA QUE** eu possa acompanhar as doações. Validar requisitos, UML, DER e script SQL.
Etapa 2	Cadastro e consulta	**COMO UM** voluntário **EU QUERO** cadastrar e consultar doações com seus itens **PARA QUE** eu mantenha um histórico local. Implementar domínio, validações e persistência.
Etapa 3	Gestão e interface	**COMO UM** voluntário **EU QUERO** editar, excluir e atualizar a situação das doações **PARA QUE** os registros reflitam as entregas. Integrar telas e resumo por situação.
Etapa 4	Validação e entrega	**COMO UM** avaliador **EU QUERO** executar o sistema com instruções claras **PARA QUE** eu verifique as funcionalidades. Testar, corrigir e revisar o README.

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

A solução utiliza **Java 17**, com tipagem estática e suporte aos pilares de **Orientação a Objetos**. O domínio emprega encapsulamento, composição, classe abstrata, herança e polimorfismo. As classes estão aninhadas em App.java para simplificar a compilação, com responsabilidades de domínio, persistência e interface separadas conceitualmente.

O banco **SQLite**, acessado por JDBC, mantém os dados em disco sem servidor externo. Chaves estrangeiras, exclusão em cascata e transações preservam a consistência entre doação e itens. O cadastro e a edição usam SQL preparado e rollback em caso de falha.

A interface **Java Swing** oferece listagem, formulário de itens, mensagens de validação e confirmação de exclusão. A situação aparece em texto. O uso é local, por um operador, sem login ou sincronização. As ações de consulta e edição partem da seleção de uma doação na tabela principal.

Viabilidade e critérios de conclusão: a execução local dispensa hospedagem e servidor de banco dedicado. Cada etapa deve manter coerência entre requisitos, classes e tabelas. O núcleo funcional é composto por RF01–RF05; RF06–RF07 complementam o acompanhamento. A conclusão de uma funcionalidade exige operação persistida e instruções de uso atualizadas.

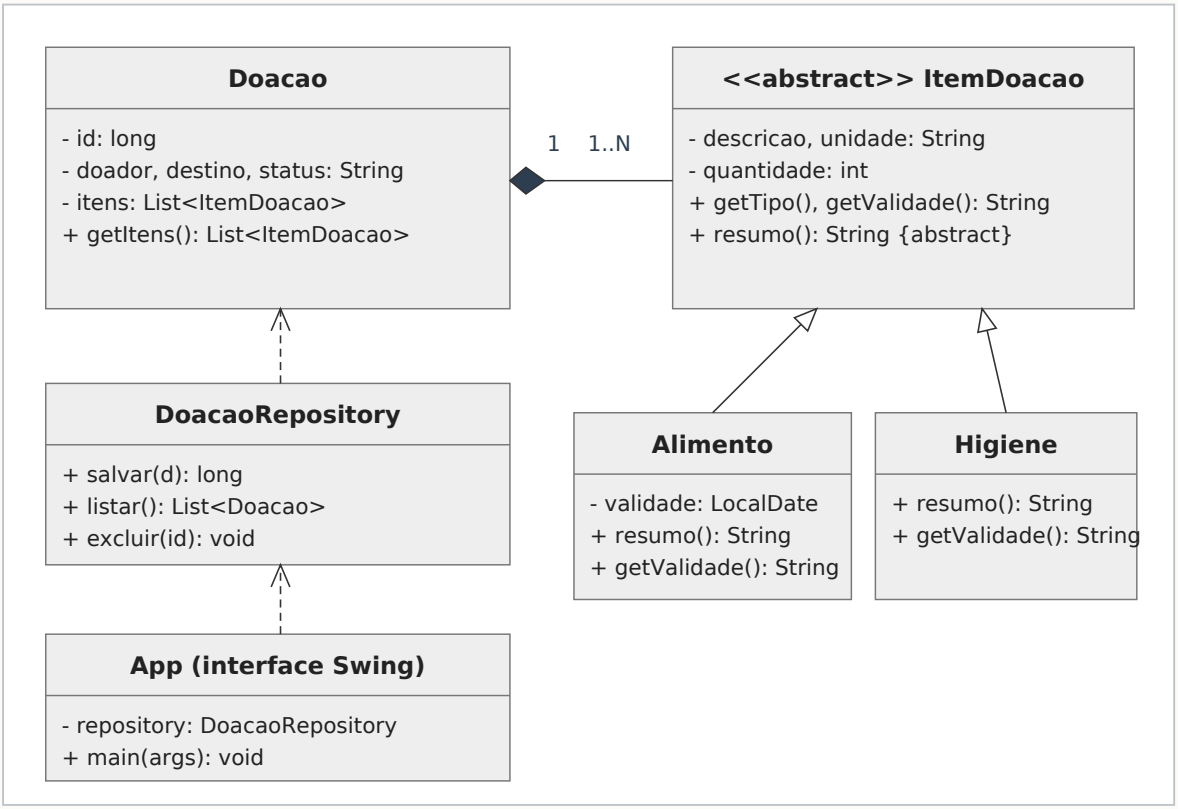
5. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

5.1 Link do Repositório

• **Repositório Público GitHub:** https://github.com/Jppjorgensen/-AEP-DoaAcao.

Pastas: **/src**, **/docs**, **/database** e **/lib**. O README e os scripts da raiz orientam a compilação e a execução.

5.2 Diagrama de Classes (UML)



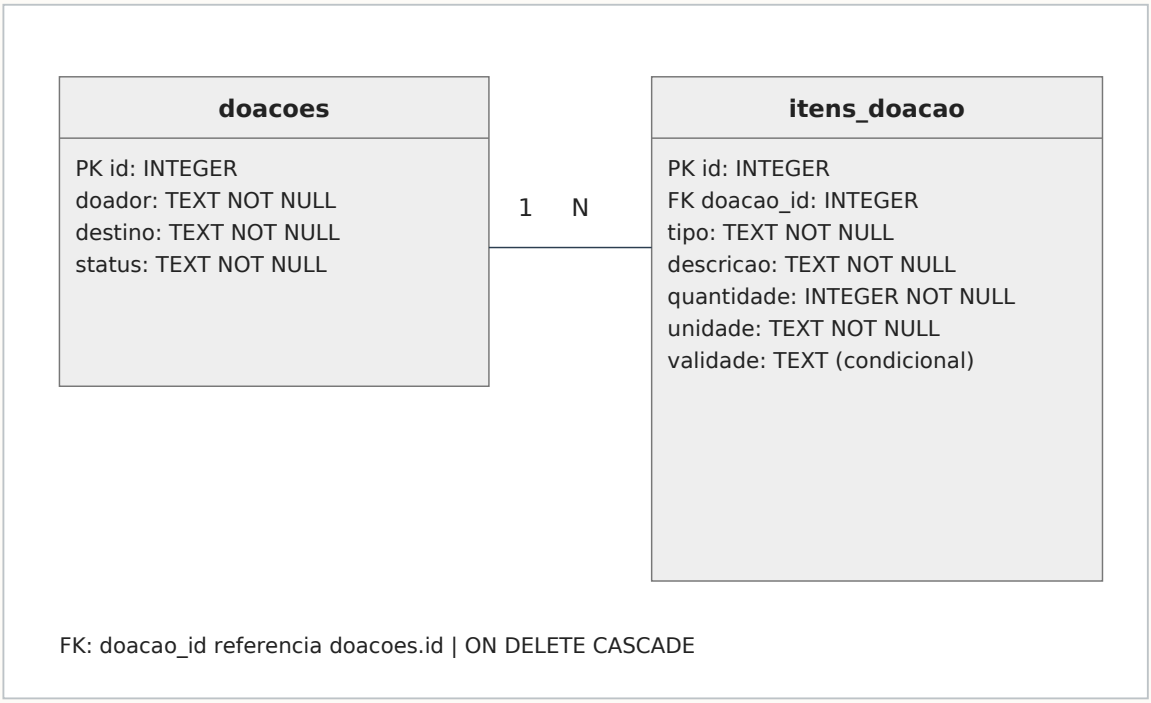
• **Herança:** Alimento e Higiene derivam da classe abstrata ItemDoacao, que reúne descrição, quantidade e unidade.

• **Composição 1:N:** cada Doacao contém ao menos um ItemDoacao. O item pertence a uma única doação e é removido com ela. O losango preenchido representa essa composição.

• **Polimorfismo:** os subtipos sobrescrevem resumo() com @Override. A confirmação de salvamento chama o método por uma referência ItemDoacao: alimentos exibem validade; itens de higiene exibem orientação de conferência da embalagem.

• **Responsabilidades:** DoacaoRepository cuida do acesso ao banco; App coordena as telas. Triângulos indicam herança e setas tracejadas indicam dependências de uso.

5.3 Diagrama de Banco de Dados (DER)



• **Tabela doacoes:** armazena a identificação do doador ou grupo, a instituição destinatária e a situação RECEBIDA ou ENTREGUE.

• **Tabela itens_doacao:** registra os itens e a chave estrangeira doacao_id. O campo tipo distingue ALIMENTO e HIGIENE. A herança do domínio é representada em uma única tabela de itens.

• **Integridade:** cada item pertence a exatamente uma doação. Quantidade deve ser inteira e positiva. Validade é obrigatória para alimentos e nula para higiene. A aplicação exige ao menos um item e salva o conjunto em uma transação.

5.4 Escopo e validação

Não há cadastros independentes de doadores ou instituições, estoque, beneficiários, relatórios exportados ou registro das datas de recebimento e entrega. A conferência de vencimento e aptidão para doação é manual. Não se somam quantidades com unidades diferentes.

Testes automatizados: cadastro, consulta, atualização, exclusão em cascata, persistência após reconexão, polimorfismo e rejeição de entradas inválidas. O comando App --test executa as verificações em um banco temporário separado. A cobertura automatizada não inclui a interação com as telas.

Ambiente e dados: a execução requer Java 17 ou superior e interface gráfica. O banco é criado automaticamente em data/doaacao.db. A cópia de segurança deve ser realizada com o programa fechado.

Referências: Guia Extraoficial da AEP — 4º semestre, 2026.2; Projeto Chamada Smart — Documento de Arquitetura e Requisitos, utilizado como referência de estrutura.